

🌟 Advanced JavaScript for React — আধুনিক ফ্রন্টএন্ডের মূল চালিকাশক্তি

React শেখা মানে মূলত আধুনিক JavaScript-এর অ্যাডভান্সড ফিচার ও প্যাটার্নসমূহ আয়ত্ত করা। শুধু JSX বা useState() জানলেই চলে না—ইমিউটেবল স্টেট আপডেট, ক্লোজার, হাইয়ার-অর্ডার ফাংশন, রেফারেন্স বনাম ভ্যালু, অ্যারে ট্রান্সফর্মেশন (map/filter/reduce), অপশনাল চেইনিং, ইভেন্ট ডেলিগেশন এবং ডিবাউন্সিং/থ্রটলিং জানা অপরিহার্য। এই চ্যাপ্টারে আমরা এই প্রতিটি বিষয় বাস্তব কোডসহ বিশদভাবে শিখব।

🔥 1. Destructuring & Spread

🔥 2. map() & filter()

🔥 3. Immutability & State

🔥 4. Reference vs Value

🔥 5. Closures in React

🔥 6. Async/Await in React

🔥 7. Synthetic Events

🔥 8. Debounce & Throttle

🔥 9. Nested State Updates

🔥 10. Common React Mistakes

Part 01 — Modern Syntax in React: Destructuring, Spread & Rest (38.1 – 38.5)

38.1 React-এর জন্য Advanced JavaScript কেন?

React মূলত JavaScript-এর উপর তৈরি। তাই React-এ এই ধরনের code খুব common:

JAVASCRIPT Source

JAVASCRIPT

```
const users = [
  { name: "Ripon", age: 23 },
  { name: "Rahim", age: 22 }
];

const names = users.map(user => user.name);
```

এখানে React নেই, কিন্তু React application-এ এই ধরনের JavaScript প্রতিদিন ব্যবহার করতে হবে।

🔥 সহজভাবে:

🌟 FULL-STACK JAVASCRIPT TO REACT PROGRESSION

```
JavaScript
↓
Modern JavaScript
↓
React
↓
Next.js / Full Stack
```

তাই Advanced JavaScript ভালোভাবে জানা = React অনেক সহজ হয়ে যাওয়া।

38.2 Destructuring

Destructuring হলো object বা array থেকে প্রয়োজনীয় value সরাসরি variable-এ বের করে নেওয়া।

🔥 Object Destructuring

JAVASCRIPT Source

JAVASCRIPT

```
const user = {  
  name: "Ripon",  
  age: 23,  
  city: "Dhaka"  
};  
  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```

♦ Output

🔥 REACT ARCHITECTURAL DIAGRAM

Ripon
23

আগে লিখতে হতো:

JAVASCRIPT Source

JAVASCRIPT

```
const name = user.name;  
const age = user.age;
```

Destructuring:

JAVASCRIPT Source

JAVASCRIPT

```
const { name, age } = user;
```

♦ React Example

JSX Source

JSX

```
function UserCard({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>{age}</p>  
    </div>  
  );  
}
```

এখানে:

JAVASCRIPT Source

JAVASCRIPT

```
{name, age}
```

হলো props destructuring।

38.3 Array Destructuring

JAVASCRIPT Source

JAVASCRIPT

```
const colors = ["red", "green", "blue"];

const [first, second, third] = colors;

console.log(first);
console.log(second);
console.log(third);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

```
red
green
blue
```

React-এর সবচেয়ে famous example:

JAVASCRIPT Source

JAVASCRIPT

```
const [count, setCount] = useState(0);
```

এখানে:

✦ REACT ARCHITECTURAL DIAGRAM

```
count    → প্রথম value
setCount → দ্বিতীয় value
```

অর্থাৎ React-এর useState() বুঝতেও Array Destructuring জানা দরকার।

38.4 Spread Operator ...

Spread operator একটি array/object-এর elements বা properties ছড়িয়ে দেয়।

🌟 Array

JAVASCRIPT Source

JAVASCRIPT

```
const numbers = [1, 2, 3];

const newNumbers = [...numbers, 4, 5];

console.log(newNumbers);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

```
[1, 2, 3, 4, 5]
```

🌟 Object

JAVASCRIPT Source

JAVASCRIPT

```
const user = {
  name: "Ripon",
  age: 23
};

const updatedUser = {
  ...user,
  age: 24
};

console.log(updatedUser);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

```
{
  name: "Ripon",
  age: 24
}
```

♦ React-এ অত্যন্ত গুরুত্বপূর্ণ

JAVASCRIPT Source

JAVASCRIPT

```
setUser({
  ...user,
  age: 24
});
```

এর অর্থ:

💡 আগের user-এর সব property রাখো, শুধু age পরিবর্তন করো।

38.5 Rest Operator

দেখতে একই:

JAVASCRIPT Source

JAVASCRIPT

...

কিন্তু কাজ আলাদা।

♦ Rest = বাকি value সংগ্রহ করা

JAVASCRIPT Source

JAVASCRIPT

```
const numbers = [1, 2, 3, 4, 5];

const [first, second, ...rest] = numbers;

console.log(first);
console.log(second);
console.log(rest);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

```
1
2
[3, 4, 5]
```

♦ Function

JAVASCRIPT Source

JAVASCRIPT

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(10, 20, 30));
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

```
60
```

38.6 `map()` — React-এর সবচেয়ে গুরুত্বপূর্ণ Array Method

`map()` array-এর প্রতিটি element নিয়ে নতুন array তৈরি করে।

JAVASCRIPT Source

JAVASCRIPT

```
const numbers = [1, 2, 3];

const doubled = numbers.map(num => num * 2);

console.log(doubled);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

[2, 4, 6]

✦ React-এ `map()`

JSX Source

JSX

```
const users = [
  { id: 1, name: "Ripon" },
  { id: 2, name: "Rahim" },
  { id: 3, name: "Karim" }
];

function Users() {
  return (
    <div>
      {users.map(user => (
        <p key={user.id}>
          {user.name}
        </p>
      ))}
    </div>
  );
}
```

♦ Browser

✦ REACT ARCHITECTURAL DIAGRAM

Ripon
Rahim
Karim

♦ এখানে:

JAVASCRIPT Source

JAVASCRIPT

```
users.map(...)
```

মানে:

💡 প্রত্যেক user-এর জন্য একটি UI element তৈরি করে।

⚡ 🔥 MUST KNOW

React-এ array render করার সময় সাধারণত:

JSX Source

JSX

```
array.map(...)
```

ব্যবহার করা হয়। এবং সাধারণত unique:

JSX Source

JSX

```
key={item.id}
```

দেওয়া হয়।

38.7 filter()

filter() condition অনুযায়ী নতুন array তৈরি করে।

JAVASCRIPT Source

JAVASCRIPT

```
const numbers = [1, 2, 3, 4, 5];

const evenNumbers = numbers.filter(num => num % 2 === 0);

console.log(evenNumbers);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

[2, 4]

♦ React Example

JSX Source

JSX

```
const products = [
  { name: "Laptop", price: 80000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 }
];

const expensiveProducts = products.filter(
  product => product.price > 5000
);
```

Result:

✦ REACT ARCHITECTURAL DIAGRAM

Laptop

38.8 find()

প্রথম matching element return করে।

JAVASCRIPT Source

JAVASCRIPT

```
const users = [
  { id: 1, name: "Ripon" },
  { id: 2, name: "Rahim" }
];

const user = users.find(user => user.id === 2);

console.log(user);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

{ id: 2, name: "Rahim" }

♦ filter() बनाम find()

Method	Result
find()	প্রথম matching element
filter()	সব matching element

38.9 findIndex()

JAVASCRIPT Source

JAVASCRIPT

```
const users = [
  { id: 10, name: "A" },
  { id: 20, name: "B" },
  { id: 30, name: "C" }
];

const index = users.findIndex(user => user.id === 20);

console.log(index);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

1

38.10 some()

কমপক্ষে একজন condition satisfy করলে true।

JAVASCRIPT Source

JAVASCRIPT

```
const ages = [15, 17, 22];

const hasAdult = ages.some(age => age >= 18);

console.log(hasAdult);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

true

◆ Real-life

💡 Class-এর মধ্যে অন্তত একজন student কি উপস্থিত?

JAVASCRIPT Source

JAVASCRIPT

```
students.some(student => student.present)
```

38.11 every()

সবাই condition satisfy করলে true।

JAVASCRIPT Source

JAVASCRIPT

```
const ages = [20, 25, 30];

const allAdults = ages.every(age => age >= 18);

console.log(allAdults);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

true

38.12 reduce()

reduce() অনেকগুলো value থেকে একটি final result তৈরি করতে পারে।

JAVASCRIPT Source

JAVASCRIPT

```
const numbers = [10, 20, 30];

const total = numbers.reduce(
  (sum, number) => sum + number,
  0
);

console.log(total);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

60

♦ Shopping Cart

JAVASCRIPT Source

JAVASCRIPT

```
const cart = [
  { name: "Laptop", price: 80000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 }
];

const total = cart.reduce(
  (sum, item) => sum + item.price,
  0
);

console.log(total);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

84000

React e-commerce app-এ এই ধরনের code খুব common।

Part 03 — Immutability, State Updates & Reference vs Value (38.13 – 38.20)

38.13 Immutability

React-এর সবচেয়ে গুরুত্বপূর্ণ conceptগুলোর একটি। **Immutability** = existing data সরাসরি পরিবর্তন না করা। ❌ Bad:

JAVASCRIPT Source

JAVASCRIPT

```
user.name = "Rahim";
```

Better:

JAVASCRIPT Source

JAVASCRIPT

```
setUser({
  ...user,
  name: "Rahim"
});
```

38.14 কেন React-এ Immutability গুরুত্বপূর্ণ?

React state update detect করতে reference comparison ব্যবহার করে। ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const user = {  
  name: "Ripon"  
};
```

সরাসরি:

JAVASCRIPT Source

JAVASCRIPT

```
user.name = "Rahim";
```

করলে একই object reference থাকে। কিন্তু:

JAVASCRIPT Source

JAVASCRIPT

```
const newUser = {  
  ...user,  
  name: "Rahim"  
};
```

এখানে নতুন object তৈরি হয়।

🌟 REACT ARCHITECTURAL DIAGRAM

```
Old Object  
  ↓  
  copy  
  ↓  
New Object
```

React state management-এর জন্য এই pattern অত্যন্ত গুরুত্বপূর্ণ।

38.15 Array State Update

ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const [items, setItems] = useState([]);
```

নতুন item যোগ:

JAVASCRIPT Source

JAVASCRIPT

```
setItems([  
  ...items,  
  newItem  
]);
```

✗ সরাসরি:

JAVASCRIPT Source

JAVASCRIPT

```
items.push(newItem);
```

এড়িয়ে চলা উচিত।

38.16 Array Item Update

ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const users = [
  { id: 1, name: "Ripon" },
  { id: 2, name: "Rahim" }
];
```

id = 2 update করতে:

JAVASCRIPT Source

JAVASCRIPT

```
const updatedUsers = users.map(user =>
  user.id === 2
    ? { ...user, name: "Karim" }
    : user
);
```

Result:

JAVASCRIPT Source

JAVASCRIPT

```
[
  { id: 1, name: "Ripon" },
  { id: 2, name: "Karim" }
]
```

🔥 React-এ এটি খুব গুরুত্বপূর্ণ pattern।

38.17 Array Item Delete

JAVASCRIPT Source

JAVASCRIPT

```
setUsers(
  users.filter(user => user.id !== 2)
);
```

অর্থ:

💡 id 2 ছাড়া বাকি সবাই রাখো।

38.18 Reference vs Value

Primitive:

JAVASCRIPT Source

JAVASCRIPT

```
let a = 10;
let b = a;

b = 20;

console.log(a);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

10

Object:

JAVASCRIPT Source

JAVASCRIPT

```
const user1 = {
  name: "Ripon"
};

const user2 = user1;

user2.name = "Rahim";

console.log(user1.name);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

Rahim

কারণ দুটো একই object reference ধরে রেখেছে।

38.19 Shallow Copy

JAVASCRIPT Source

JAVASCRIPT

```
const user = {  
  name: "Ripon",  
  age: 23  
};  
  
const copy = {  
  ...user  
};
```

এখন:

★ REACT ARCHITECTURAL DIAGRAM

user ~~≡~~ copy

কিন্তু nested object থাকলে সমস্যা হতে পারে।

JAVASCRIPT Source

JAVASCRIPT

```
const user = {  
  name: "Ripon",  
  address: {  
    city: "Dhaka"  
  }  
};  
  
const copy = { ...user };  
  
copy.address.city = "Dinajpur";
```

এতে original-এর nested object-ও পরিবর্তিত হতে পারে। কারণ spread **deep copy** নয়।

38.20 Deep Copy

Modern JavaScript-এ:

JAVASCRIPT Source

JAVASCRIPT

```
const copy = structuredClone(user);
```

এটি অনেক ক্ষেত্রে nested data-এর deep copy তৈরি করতে পারে।

38.21 Optional Chaining ?.

ধরো API response:

JAVASCRIPT Source

JAVASCRIPT

```
const user = {};
```

এখন:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(user.address.city);
```

error হতে পারে। কিন্তু:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(user.address?.city);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

undefined

আরও:

JAVASCRIPT Source

JAVASCRIPT

```
user?.profile?.address?.city
```

React-এ API data render করার সময় খুব usefull

38.22 Nullish Coalescing ??

JAVASCRIPT Source

JAVASCRIPT

```
const username = null;

const name = username ?? "Guest";

console.log(name);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

Guest

♦ || বনাম ??

JAVASCRIPT Source

JAVASCRIPT

```
const count = 0;

console.log(count || 10);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

10

কিন্তু:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(count ?? 10);
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

0

কারণ ?? শুধু null অথবা undefined check করে।

38.23 Short-Circuit Rendering

React:

JSX Source

JSX

```
{isLoggedIn && <Dashboard />}
```

মানে:

✦ REACT ARCHITECTURAL DIAGRAM

```
isLoggedIn = true
  ↓
Dashboard দেখাও
```

অথবা:

✦ REACT ARCHITECTURAL DIAGRAM

```
false
  ↓
কিছু render করে না
```

38.24 Ternary Operator

JAVASCRIPT Source

JAVASCRIPT

```
const isLoggedIn = true;

const message = isLoggedIn
  ? "Welcome"
  : "Please login";

console.log(message);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

Welcome

React:

JSX Source

JSX

```
{isLoggedIn ? (
  <Dashboard />
) : (
  <Login />
)}
```

এটি React-এর খুব common pattern।

38.25 Arrow Functions

JAVASCRIPT Source

JAVASCRIPT

```
const add = (a, b) => {
  return a + b;
};
```

Short form:

JAVASCRIPT Source

JAVASCRIPT

```
const add = (a, b) => a + b;
```

React:

JSX Source

JSX

```
users.map(user => (
  <p>{user.name}</p>
))
```

38.26 Higher-Order Function

যে function:

- অন্য function গ্রহণ করে অথবা
- function return করে

তাকে Higher-Order Function বলা হয়। উদাহরণ:

JAVASCRIPT Source

JAVASCRIPT

```
function calculate(a, b, operation) {  
  return operation(a, b);  
}  
  
const result = calculate(  
  10,  
  20,  
  (a, b) => a + b  
);  
  
console.log(result);
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

30

React-এর:

JAVASCRIPT Source

JAVASCRIPT

```
map()  
filter()  
reduce()
```

এগুলো callback গ্রহণ করে।

38.27 Callback Function

JAVASCRIPT Source

JAVASCRIPT

```
function greet(name, callback) {  
  console.log("Hello", name);  
  callback();  
}  
  
greet("Ripon", () => {  
  console.log("Welcome!");  
});
```

Output:

🌟 REACT ARCHITECTURAL DIAGRAM

Hello Ripon
Welcome!

38.28 Closures in React

Closure মানে inner function তার outer scope-এর variable মনে রাখতে পারে।

JAVASCRIPT Source

JAVASCRIPT

```
function counter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const increase = counter();

console.log(increase());
console.log(increase());
```

Output:

✦ REACT ARCHITECTURAL DIAGRAM

1
2

React-এ event handler এবং hooks-এর behaviour বুঝতে closure অত্যন্ত গুরুত্বপূর্ণ।

38.29 React State এবং Closure

JSX Source

JSX

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    console.log(count);
  }

  return (
    <button onClick={handleClick}>
      {count}
    </button>
  );
}
```

handleClick তার surrounding scope থেকে count access করছে। এটাই closure-এর practical impact।

38.30 Functional Programming

React development-এ functional programming style বেশি দেখা যায়। মূল concepts:

✦ IMMUTABLE REACT STATE LIFECYCLE

Pure Function
Immutability
Higher-Order Function
Composition
Declarative Programming

38.31 Pure Function

Pure function: 1. একই input → একই output 2. বাইরে কোনো side effect করে না

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}
```

এটি pure। কিন্তু:

JAVASCRIPT Source

JAVASCRIPT

```
let total = 0;  
  
function addToTotal(value) {  
  total += value;  
}
```

এটি pure নয় কারণ external state পরিবর্তন করছে।

38.32 this in React

Modern React functional components-এ সাধারণত class-এর মতো this লাগে না। পুরনো React:

JAVASCRIPT Source

JAVASCRIPT

```
class Counter extends React.Component {  
  handleClick() {  
    this.setState({  
      count: this.state.count + 1  
    });  
  }  
}
```

Modern React:

JSX Source

JSX

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  
  );  
}
```

তাই modern React শেখার জন্য this জানা দরকার হলেও এর ব্যবহার অনেক কম।

38.33 Modules

React project সাধারণত module-based।

JAVASCRIPT Source

JAVASCRIPT

```
// math.js

export function add(a, b) {
  return a + b;
}
```

তারপর:

JAVASCRIPT Source

JAVASCRIPT

```
import { add } from "./math.js";
```

React:

JAVASCRIPT Source

JAVASCRIPT

```
import React from "react";
import UserCard from "./UserCard";
```

38.34 Default Export

JAVASCRIPT Source

JAVASCRIPT

```
export default function UserCard() {
  return <h2>User</h2>;
}
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
import UserCard from "./UserCard";
```

38.35 Named Export

JAVASCRIPT Source

JAVASCRIPT

```
export function add() {}

export function subtract() {}
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
import {
  add,
  subtract
} from "./math.js";
```

38.36 Async/Await in React

API call:

JAVASCRIPT Source

JAVASCRIPT

```

async function getUsers() {
  const response = await fetch(
    "https://example.com/users"
  );

  const data = await response.json();

  console.log(data);
}

```

React-এ সাধারণত useEffect()-এর মাধ্যমে API call করা হয়। Concept:

🔥 IMMUTABLE REACT STATE LIFECYCLE

```

Component
  ↓
useEffect
  ↓
fetch()
  ↓
await response
  ↓
JSON
  ↓
setState
  ↓
Re-render

```

38.37 Promise Handling

JAVASCRIPT Source

JAVASCRIPT

```

fetch("/api/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error(error);
  });

```

Modern style:

JAVASCRIPT Source

JAVASCRIPT

```

async function loadUsers() {
  try {
    const response = await fetch("/api/users");

    const data = await response.json();

    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

```

38.38 API Data Processing

ধরো backend থেকে এসেছে:

JAVASCRIPT Source

JAVASCRIPT

```
const users = [
  {
    id: 1,
    name: "Ripon",
    active: true
  },
  {
    id: 2,
    name: "Rahim",
    active: false
  }
];
```

শুধু active users:

JAVASCRIPT Source

JAVASCRIPT

```
const activeUsers = users.filter(
  user => user.active
);
```

তারপর names:

JAVASCRIPT Source

JAVASCRIPT

```
const names = activeUsers.map(
  user => user.name
);
```

Result:

★ REACT ARCHITECTURAL DIAGRAM

["Ripon"]

React:

JSX Source

JSX

```
{users
  .filter(user => user.active)
  .map(user => (
    <p key={user.id}>
      {user.name}
    </p>
  ))
}
```

🔥 Full-stack React development-এ এই pattern অত্যন্ত common!

38.39 Event Object

JSX Source

JSX

```
function handleClick(event) {  
  console.log(event);  
}
```

JSX Source

JSX

```
<button onClick={handleClick}>  
  Click  
</button>
```

event object-এর মধ্যে event সম্পর্কিত তথ্য থাকে। যেমন:

JAVASCRIPT Source

JAVASCRIPT

```
event.target  
event.currentTarget  
event.preventDefault()
```

38.40 preventDefault()

Form:

JSX Source

JSX

```
function handleSubmit(event) {  
  event.preventDefault();  
  
  console.log("Form submitted");  
}
```

Browser-এর default form submission বন্ধ করে।

38.41 Event Bubbling

HTML:

✦ REACT ARCHITECTURAL DIAGRAM

```
<div id="parent">  
  <button id="child">  
    Click  
  </button>  
</div>
```

Button click করলে event:

✦ REACT ARCHITECTURAL DIAGRAM

```
button  
↓  
parent  
↓  
body  
↓  
document
```

উপরে উঠতে পারে। এটিই event bubbling।

38.42 Event Delegation

অনেকগুলো child element-এর জন্য আলাদা listener না দিয়ে parent-এ listener ব্যবহার করা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
parent.addEventListener("click", event => {
  if (event.target.matches("button")) {
    console.log("Button clicked");
  }
});
```

React-এ event system এই concept-এর সাথে সম্পর্কিত।

38.43 Debounce

User typing:

✦ REACT ARCHITECTURAL DIAGRAM

R
Ri
Rip
Ripo
Ripon

প্রতিবার API call করলে unnecessary request হবে। Debounce:

💡 User typing বন্ধ করার কিছু সময় পর function চালাও।

JAVASCRIPT Source

JAVASCRIPT

```
function debounce(callback, delay) {
  let timer;

  return (...args) => {
    clearTimeout(timer);

    timer = setTimeout(() => {
      callback(...args);
    }, delay);
  };
}
```

Search:

JAVASCRIPT Source

JAVASCRIPT

```
const search = debounce(value => {
  console.log("API call:", value);
}, 500);
```

38.44 Throttle

Throttle:

💡 নির্দিষ্ট সময়ের মধ্যে সর্বোচ্চ একবার function চালাও।

যেমন:

✦ REACT ARCHITECTURAL DIAGRAM

```
scroll
scroll
scroll
scroll
scroll
```

প্রতিবার না চালিয়ে:

✦ REACT ARCHITECTURAL DIAGRAM

```
100ms
200ms
300ms
```

এরকম interval-এ চালানো যায়।

Part 07 — Advanced State Updates, Common React Pitfalls & Real Todo Architecture (38.45 – 38.54)

38.45 React State Update

ধরো:

JSX Source

JSX

```
const [count, setCount] = useState(0);
```

Wrong ধারণা:

JAVASCRIPT Source

JAVASCRIPT

```
count = count + 1;
```

State update করতে হবে:

JAVASCRIPT Source

JAVASCRIPT

```
setCount(count + 1);
```

আর previous state-এর উপর নির্ভর করলে:

JAVASCRIPT Source

JAVASCRIPT

```
setCount(prevCount => prevCount + 1);
```

♦ কেন?

কারণ React state update asynchronous/batched হতে পারে। এই pattern safer:

JAVASCRIPT Source

JAVASCRIPT

```
setCount(prev => prev + 1);
```

38.46 Multiple State Updates

এটি:

JAVASCRIPT Source

JAVASCRIPT

```
setCount(count + 1);  
setCount(count + 1);
```

সবসময় +2 করার নিশ্চয়তা দেয় না। Better:

JAVASCRIPT Source

JAVASCRIPT

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

এখানে প্রতিটি update previous state-এর উপর নির্ভর করছে।

38.47 Nested State Update

ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const [user, setUser] = useState({  
  name: "Ripon",  
  address: {  
    city: "Dhaka"  
  }  
});
```

City update:

JAVASCRIPT Source

JAVASCRIPT

```
setUser(prev => ({  
  ...prev,  
  address: {  
    ...prev.address,  
    city: "Dinajpur"  
  }  
}));
```

এখানে দুই level spread হয়েছে।

38.48 React-এ Conditional Rendering

◆ &&

JSX Source

JSX

```
{isAdmin && <AdminPanel />}
```

◆ Ternary

JSX Source

JSX

```
{isAdmin ? (  
  <AdminPanel />  
) : (  
  <UserPanel />  
)}
```

◆ Function

JSX Source

JSX

```
function renderContent() {  
  if (loading) return <Loading />;  
  if (error) return <Error />;  
  return <Content />;  
}
```

38.49 Common Mistake — map() এ return ভুলে যাওয়া

✗:

JAVASCRIPT Source

JAVASCRIPT

```
users.map(user => {  
  <p>{user.name}</p>  
});
```

এখানে implicit return নেই সঠিক:

JAVASCRIPT Source

JAVASCRIPT

```
users.map(user => (  
  <p>{user.name}</p>  
));
```

অথবা:

JAVASCRIPT Source

JAVASCRIPT

```
users.map(user => {  
  return <p>{user.name}</p>;  
});
```

38.50 Common Mistake — Object Mutation

✗:

JAVASCRIPT Source

JAVASCRIPT

```
user.name = "Rahim";
```

React state হলে:

JAVASCRIPT Source

JAVASCRIPT

```
setUser(prev => ({  
  ...prev,  
  name: "Rahim"  
}));
```

38.51 Common Mistake — Array Mutation

✗:

JAVASCRIPT Source

JAVASCRIPT

```
items.push(newItem);
```

Better:

JAVASCRIPT Source

JAVASCRIPT

```
setItems(prev => [  
  ...prev,  
  newItem  
]);
```

Delete:

JAVASCRIPT Source

JAVASCRIPT

```
setItems(prev =>  
  prev.filter(item => item.id !== id)  
);
```

Update:

JAVASCRIPT Source

JAVASCRIPT

```
setItems(prev =>  
  prev.map(item =>  
    item.id === id  
      ? { ...item, completed: true }  
      : item  
  )  
);
```

38.52 Common Mistake — Index কে সবসময় key হিসেবে ব্যবহার করা

JSX Source

JSX

```
{items.map((item, index) => (  
  <div key={index}>  
    {item.name}  
  </div>  
))}
```

Static list হলে কখনো acceptable হতে পারে। কিন্তু list যদি:

- reorder হয়
- delete হয়
- insert হয়

তাহলে stable unique ID ব্যবহার করা better:

JSX Source

JSX

```
key={item.id}
```

38.53 Common Mistake — API Data আসার আগেই Access করা

ধরো:

JSX Source

JSX

```
<p>{user.profile.name}</p>
```

API data এখনো আসেনি। Error হতে পারে। Better:

JSX Source

JSX

```
<p>{user?.profile?.name}</p>
```

অথবা loading state:

JSX Source

JSX

```
if (loading) {  
  return <p>Loading...</p>;  
}
```

ধরো:

JAVASCRIPT Source

JAVASCRIPT

```
const todos = [
  {
    id: 1,
    title: "Learn JavaScript",
    completed: true
  },
  {
    id: 2,
    title: "Learn React",
    completed: false
  }
];
```

◆ Completed todos

JAVASCRIPT Source

JAVASCRIPT

```
const completedTodos = todos.filter(
  todo => todo.completed
);
```

◆ Todo names

JAVASCRIPT Source

JAVASCRIPT

```
const titles = todos.map(
  todo => todo.title
);
```

◆ Update

JAVASCRIPT Source

JAVASCRIPT

```
const updatedTodos = todos.map(todo =>
  todo.id === 2
    ? { ...todo, completed: true }
    : todo
);
```

◆ Delete

JAVASCRIPT Source

JAVASCRIPT

```
const remainingTodos = todos.filter(
  todo => todo.id !== 1
);
```

এই চারটি pattern React Todo App-এর core।

38.55 🔥 MUST KNOW — React-এর জন্য অপরিহার্য JavaScript কনসেপ্ট

সবচেয়ে বেশি গুরুত্ব দাও:

Topic	Importance
map()	🔥🔥🔥
filter()	🔥🔥🔥
Destructuring	🔥🔥🔥
Spread	🔥🔥🔥
Immutability	🔥🔥🔥
Arrow Function	🔥🔥🔥
Callback	🔥🔥🔥
Closure	🔥🔥
reduce()	🔥🔥
Optional Chaining	🔥🔥🔥
Nullish Coalescing	🔥🔥
Ternary	🔥🔥🔥
Async/Await	🔥🔥🔥
Promise	🔥🔥🔥
Modules	🔥🔥🔥
Reference	🔥🔥🔥
Event Object	🔥🔥
Debounce	🔥🔥
Throttle	🔥🔥

JAVASCRIPT Source

JAVASCRIPT

```
// Destructuring
const { name } = user;

// Array destructuring
const [first, second] = items;

// Spread
const newArray = [...oldArray];

// Object spread
const newUser = { ...user, age: 24 };

// Rest
const [first, ...rest] = numbers;

// Map
items.map(item => item.name);

// Filter
items.filter(item => item.active);

// Find
items.find(item => item.id === 1);

// Some
items.some(item => item.active);

// Every
items.every(item => item.active);

// Reduce
items.reduce((sum, item) => sum + item.price, 0);

// Optional chaining
user?.profile?.name;

// Nullish
name ?? "Guest";

// Ternary
isLoggedIn ? <Dashboard /> : <Login />;

// Short circuit
isAdmin && <AdminPanel />;

// Immutable update
setUser(prev => ({
  ...prev,
  name: "Rahim"
}));

// Immutable array add
setItems(prev => [...prev, item]);

// Immutable delete
setItems(prev =>
  prev.filter(item => item.id !== id)
);

// Immutable update
setItems(prev =>
  prev.map(item =>
    item.id === id
      ? { ...item, done: true }
      : item
  )
);
```

Lab 1: Array Filtering — Extracting Adults (Age >= 18)

BEGINNER

সমস্যা ও লক্ষ্য: প্রদত্ত ইউজার অ্যারে থেকে filter() মেথড ব্যবহার করে শুধুমাত্র ১৮ বা তার বেশি বয়সের ইউজারদের ফিল্টার করুন।

PRODUCTION SOLUTION: LAB 1

JAVASCRIPT

```
const users = [
  { name: "A", age: 15 },
  { name: "B", age: 20 },
  { name: "C", age: 25 }
];

// Modern ES6+ / React Pattern
const adultUsers = users.filter(user => user.age >= 18);

console.log(adultUsers);
// Output: [ { name: 'B', age: 20 }, { name: 'C', age: 25 } ]
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** filter() মূল অ্যারে মিউটেট না করে একটি সম্পূর্ণ নতুন অ্যারে তৈরি করে, যা React-এর স্টেট ফিল্টারিং ও সার্চ ফিচারে শতভাগ নিরাপদ।

Lab 2: Array Projection — Mapping Array of Names

BEGINNER

সমস্যা ও লক্ষ্য: ইউজার অবজেক্ট অ্যারে থেকে map() ব্যবহার করে শুধুমাত্র ইউজারদের নামের একটি স্ট্রিং অ্যারে তৈরি করুন।

PRODUCTION SOLUTION: LAB 2

JAVASCRIPT

```
const users = [
  { name: "A", age: 15 },
  { name: "B", age: 20 },
  { name: "C", age: 25 }
];

// React Projection Pattern
const userNames = users.map(user => user.name);

console.log(userNames);
// Output: ["A", "B", "C"]
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** map() মেথডটি মূল অ্যারের প্রতিটি উপাদান রূপান্তর করে সমপরিমাণ উপাদানের নতুন অ্যারে বানায়—যা React-এ ডেটা থেকে JSX এলিমেন্ট রেন্ডারিংয়ের মূল ভিত্তি।

Lab 3: Immutable Item Update — Updating Object by ID

BEGINNER

সমস্যা ও লক্ষ্য: map() এবং Spread Operator ব্যবহার করে id = 2 ইউজারের নাম পরিবর্তন করে 'Ripon' করুন (অন্যান্য উপাদান অক্ষত রেখে)।

PRODUCTION SOLUTION: LAB 3

JAVASCRIPT

```
const users = [
  { id: 1, name: "Tanvir" },
  { id: 2, name: "OldName" },
  { id: 3, name: "Karim" }
];

// Immutable Update Pattern
const targetId = 2;
const updatedUsers = users.map(user =>
  user.id === targetId
    ? { ...user, name: "Ripon" }
    : user
);

console.log(updatedUsers);
// Output: [ { id: 1, name: 'Tanvir' }, { id: 2, name: 'Ripon' }, { id: 3, name: 'Karim' } ]
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** React স্টেটে কোনো অবজেক্ট সরাসরি মিউটেট করা নিষিদ্ধ। টার্নারি অপারেটর এবং অবজেক্ট স্প্রেড({...user}) দিয়ে টার্গেট আইটেমটির কপি তৈরি করে আপডেট করতে হয়।

Lab 4: Immutable Item Deletion — Removing Object by ID

BEGINNER

সমস্যা ও লক্ষ্য: `filter()` মেথড ব্যবহার করে `id = 3` ইউজারটিকে অ্যারে থেকে নিরাসদে মুছে ফেলুন।

PRODUCTION SOLUTION: LAB 4

JAVASCRIPT

```
const users = [
  { id: 1, name: "Tanvir" },
  { id: 2, name: "Ripon" },
  { id: 3, name: "Karim" }
];

// Immutable Delete Pattern
const deleteId = 3;
const remainingUsers = users.filter(user => user.id !== deleteId);

console.log(remainingUsers);
// Output: [ { id: 1, name: 'Tanvir' }, { id: 2, name: 'Ripon' } ]
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** React-এ Delete অপারেশনের জন্য `splice()` ব্যবহার করলে মূল অ্যারে পরিবর্তিত হয়ে যায়। `filter()` মেথড শর্ত না মেলা আইটেমগুলো বাদ দিয়ে নতুন অ্যারে দেয়।

Lab 5: Array Aggregation — Shopping Cart Total via `reduce()`

INTERMEDIATE

সমস্যা ও লক্ষ্য: `reduce()` মেথড ব্যবহার করে শপিং কার্টের সকল আইটেমের মোট মূল্য (Total Price) বের করুন।

PRODUCTION SOLUTION: LAB 5

JAVASCRIPT

```
const cart = [
  { name: "Laptop", price: 80000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 }
];

// React / E-Commerce Accumulator Pattern
const totalPrice = cart.reduce((accumulator, item) => accumulator + item.price, 0);

console.log('Total Cart Amount: ₳' + totalPrice);
// Output: Total Cart Amount: ₳84000
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** `reduce()` মেথডে প্রারম্ভিক মান (Initial Value: 0) উল্লেখ করা বাধ্যতামূলক। এটি কার্ট টোটাল, ট্যাক্স বা ডিসকাউন্ট ক্যালকুলেশনের স্ট্যান্ডার্ড পদ্ধতি।

Lab 6: React Component Bug Fix — Missing Return in map()

INTERMEDIATE

সমস্যা ও লক্ষ্য: নিচের কম্পোনেন্টটিতে map() এ ব্রাকেট দেওয়ার কারণে JSX রিটার্ন হচ্ছে না। এরর খুঁজে বের করে প্রোডাকশন-রেডি ফিক্স লিখুন।

PRODUCTION SOLUTION: LAB 6

JAVASCRIPT

```
// ❌ Buggy Version (Returns undefined for each item)
// function Users({ users }) {
//   return (<div>{users.map(user => { <p>{user.name}</p> })}</div>);
// }

// ✅ Fixed Version 1: Implicit Return with Parentheses
function UsersImplicit({ users }) {
  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

// ✅ Fixed Version 2: Explicit Return Keyword with Curly Braces
function UsersExplicit({ users }) {
  return (
    <div>
      {users.map(user => {
        return <p key={user.id}>{user.name}</p>;
      })}
    </div>
  );
}
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** অ্যারো ফাংশনে কার্লি ব্রেস {} দিলে বডি তৈরি হয়, ফলে রিটার্ন করতে হলে 'return' কিওয়ার্ড লিখতে হয়। অথবা প্যারেন্থেসিস () দিয়ে ইমপ্লিসিট রিটার্ন করতে হয়। সাথে অনন্য key প্রপ যুক্ত করা জরুরি।

Lab 7: Functional State Update — Immutable Top-Level Property

INTERMEDIATE

সমস্যা ও লক্ষ্য: React Functional setState প্যাটার্ন মেনে user অবজেক্টের age প্রপাটি ২৩ থেকে ২৪ এ আপডেট করুন।

PRODUCTION SOLUTION: LAB 7

JAVASCRIPT

```
const [user, setUser] = [
  { name: "Ripon", age: 23 },
  newUser => console.log('Updated User State:', newUser)
];

// React Functional State Update Pattern
setUser(prevUser => ({
  ...prevUser,
  age: 24
}));

// Output: Updated User State: { name: 'Ripon', age: 24 }
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** setState-এ ফাংশনাল আপডেট (prev => ({...prev, prop})) ব্যবহার করলে রেস কন্ডিশন ও স্টেল স্টেট (Stale State) এরর সম্পূর্ণ দূর হয়।

Lab 8: Deep Immutable Update — Nested Object State Modification

ADVANCED

সমস্যা ও লক্ষ্য: Nested অবজেক্টের গভীরের প্রপার্টি (address.city) মিউটেট না করে স্প্রেড অপারেটর দিয়ে 'Dhaka' থেকে 'Dinajpur' এ আপডেট করুন।

PRODUCTION SOLUTION: LAB 8

JAVASCRIPT

```
const user = {
  name: "Ripon",
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
};

// ❌ Buggy Shallow Spread: user.address মিউটেট হয়ে যায়
// const wrong = { ...user, address: { city: 'Dinajpur' } }; // country হারিয়ে যাবে!

// ✅ Correct Deep Immutable Spread
const updatedUser = {
  ...user,
  address: {
    ...user.address,
    city: "Dinajpur"
  }
};

console.log(updatedUser);
// Output: { name: 'Ripon', address: { city: 'Dinajpur', country: 'Bangladesh' } }
```

💡 **রিঅ্যাক্ট আর্কিটেকচারাল ব্যাখ্যা:** Shallow Spread কেবল প্রথম লেভেলের প্রপার্টি কপি করে। নেস্টেড অবজেক্ট থাকলে ভেতরের অবজেক্টকেও আলাদাভাবে স্প্রেড ({...user.address}) করতে হয়, নতুবা অবজেক্টের অন্যান্য প্রপার্টি মুছে যাবে।

38.57

🗺️ Final Mental Map — React-এর জন্য JavaScript আর্কিটেকচার মানচিত্র

পুরো Chapter 38-এর JavaScript ও React সমন্বয় একনজরে:

🌟 REACT JAVASCRIPT TAXONOMY MAP

Advanced JS for React

Syntax

Destructuring
Spread/Rest
Arrow Function

Data

map/filter
find/reduce
some/every

Logic

ternary
?? / ??
optional chaining

React State

Immutability

Add

spread

Update

map

Delete

filter

API / Async

Promise / await

React UI

🔥 **এক লাইনে Chapter 38:** React ফ্রেমওয়ার্ক শেখার প্রথম ও প্রধান ভিত্তি হলো আধুনিক JavaScript-এর Destructuring, Spread, map/filter/reduce, Immutability এবং Async/Await—এগুলো আয়ত্ত করলেই React কোড পানির মতো সহজ মনে হবে।

🌟 ROADMAP CONTINUITY PIPELINE

Chapter 37: JavaScript Design Patterns



Creational, Structural, Behavioral, MVC, Dependency Injection & Repository



Chapter 38: Advanced JavaScript for **React** (THIS CHAPTER)



Destructuring, Spread, **map/filter**, **Immutability**, Closures, Synthetic Events



Chapter 39: Advanced JavaScript for Node.js



Node.js Architecture, Libuv, Event Loop Phases, Streams, Buffer, Worker Threads